

叉车式移动机器人完整仿真系统研究报告

刘振毫

学号：2312167

专业：智能科学与工程

2026 年 5 月 27 日

摘要

本文针对叉车式移动机器人的自主导航问题，系统地研究了其运动学建模、里程计模型、控制算法以及基于卡尔曼滤波的定位算法。首先，采用瞬心法和运动约束法两种方法构建了叉车的运动学模型，分析了其非完整约束特性。其次，建立了里程计模型和误差传导模型，实现了对机器人位姿的连续估计。然后，设计了基于Lyapunov稳定性的轨迹跟踪控制器，实现了对参考轨迹的稳定跟踪。最后，实现了扩展卡尔曼滤波(EKF)定位算法，通过融合里程计和激光雷达观测信息，显著提高了定位精度。

仿真实验结果表明，EKF定位算法相比纯里程计方法，在X方向、Y方向和航向角的定位精度分别提高了84.9%、77.2%和49.3%，验证了所提方法的有效性。本研究为叉车式移动机器人的自主导航提供了完整的理论框架和实现方案。

关键词：叉车式移动机器人；运动学建模；里程计；卡尔曼滤波；自主定位

目录

摘要	1
1 实验介绍	7
1.1 实验设计	7
1.1.1 实验任务设计	7
1.1.2 实验流程设计	7
1.2 实验环境	7
1.2.1 硬件环境	7
1.2.2 软件环境	8
1.2.3 仿真参数配置	8
1.2.4 机器人参数配置	8
1.2.5 噪声模型参数	9
2 系统建模	10
2.1 叉车模型描述	10
2.2 瞬心法建模	10
2.2.1 瞬时旋转中心	10
2.2.2 运动学方程	11
2.3 运动约束法建模	11
2.3.1 非完整约束	11
2.3.2 约束矩阵	11
2.3.3 可行运动空间	12
2.4 两种方法对比	12
3 控制算法设计	13
3.1 轨迹跟踪问题描述	13
3.2 Lyapunov控制器设计	13
3.2.1 误差模型	13
3.2.2 控制律设计	13
3.2.3 稳定性分析	13
3.3 转向角计算	14
3.4 控制参数选择	14
4 里程计模型	15
4.1 里程计基本原理	15

4.2	位姿更新方程	15
4.3	误差传导模型	15
4.3.1	雅可比矩阵	15
4.3.2	协方差传播	16
4.4	误差特性分析	16
5	基于激光雷达的2D点云直线提取算法	17
5.1	算法原理	17
5.1.1	Split-and-Merge算法	17
5.1.2	直线模型	17
5.2	算法实现	17
5.2.1	算法流程	17
5.2.2	关键参数	18
5.3	仿真实验	18
5.3.1	实验设置	18
5.3.2	实验结果	18
6	基于激光雷达的2D点云配准算法	19
6.1	ICP算法原理	19
6.1.1	迭代最近点算法	19
6.1.2	刚体变换	19
6.2	算法实现	19
6.2.1	对应点查找	19
6.2.2	变换估计	19
6.2.3	收敛判断	20
6.3	仿真实验	20
6.3.1	实验设置	20
6.3.2	实验结果	20
7	卡尔曼滤波定位算法	21
7.1	扩展卡尔曼滤波原理	21
7.2	状态空间模型	21
7.2.1	状态向量	21
7.2.2	运动模型	21
7.2.3	观测模型	21
7.3	EKF算法流程	22
7.3.1	预测步骤	22
7.3.2	更新步骤	22
7.4	雅可比矩阵计算	22
7.4.1	运动雅可比矩阵	22

7.4.2	观测雅可比矩阵	23
8	仿真实验与结果分析	24
8.1	仿真环境设置	24
8.1.1	仿真参数	24
8.1.2	叉车模型参数	24
8.2	参考轨迹生成	24
8.3	定位性能分析	25
8.3.1	轨迹跟踪结果	25
8.3.2	定位误差分析	26
8.3.3	定位精度统计	29
8.4	不确定性分析	30
8.4.1	协方差演化	30
8.4.2	不确定性椭圆	31
8.5	控制性能分析	32
8.5.1	速度控制输入	32
8.5.2	转向角控制输入	32
A	附录	34
A.1	源代码	34
B	符号说明	50

插图

8.1	机器人轨迹对比	25
8.2	X方向定位误差	26
8.3	Y方向定位误差	27
8.4	航向角定位误差	28
8.5	总位置误差对比	29
8.6	不确定性标准差演化	30
8.7	不确定性椭圆演化	31
8.8	速度控制输入	32
8.9	转向角控制输入	33

表格

1.1	仿真参数配置	8
1.2	机器人参数配置	8
1.3	噪声模型参数	9
8.1	仿真参数设置	24
8.2	叉车模型参数	24
8.3	定位性能对比	29
B.1	符号说明表	50

第一章 实验介绍

第一节 实验设计

1.1.1 实验任务设计

本实验旨在构建一个完整的叉车式移动机器人自主导航系统，实现机器人在未知环境中的精确定位和轨迹跟踪。具体实验任务包括：

1. **系统建模任务：**分别采用瞬心法和运动约束法构建叉车式移动机器人的运动学模型，分析其非完整约束特性，为后续控制算法和定位算法设计提供理论基础。
2. **控制算法设计任务：**设计基于Lyapunov稳定性的轨迹跟踪控制器，实现机器人对参考轨迹的稳定跟踪，并通过仿真实验验证控制器的性能。
3. **里程计模型分析任务：**建立机器人的里程计模型和误差传导模型，分析里程计误差的累积特性，为定位算法提供预测模型。
4. **点云处理任务：**设计并实现基于激光雷达的2D点云直线提取算法和ICP点云配准算法，为环境感知和特征提取提供技术支持。
5. **定位算法实现任务：**设计并实现基于扩展卡尔曼滤波(EKF)的机器人定位算法，融合里程计和激光雷达观测信息，实现高精度定位。
6. **综合仿真验证任务：**整合所有模块，设计完整的仿真实验，验证所提方法的有效性和实用性。

1.1.2 实验流程设计

实验采用模块化设计思想，按照以下流程进行：

1. 环境建模：生成包含地标特征的仿真环境
2. 轨迹生成：设计参考轨迹和控制输入
3. 运动仿真：基于运动学模型模拟机器人运动
4. 传感器仿真：模拟里程计和激光雷达的观测数据
5. 算法实现：实现控制、定位等核心算法
6. 性能评估：对比分析不同方法的定位精度

第二节 实验环境

1.2.1 硬件环境

本实验采用仿真方式进行，主要硬件配置如下：

1. 处理器：12th Gen Intel(R) Core(TM) i5-1240P (1.70 GHz)
2. 内存：16GB
3. 存储：512GB SSD
4. 操作系统：Windows 11

1.2.2 软件环境

实验使用的软件环境包括：

1. 编程语言：Python 3.8
2. 数值计算库：NumPy 1.20, SciPy 1.7
3. 可视化库：Matplotlib 3.3
4. 开发环境：Visual Studio Code

1.2.3 仿真参数配置

实验中使用的仿真参数配置如表8.1所示。

表 1.1: 仿真参数配置

参数名称	参数值	说明
仿真时间	40.0 s	总仿真时长
时间步长	0.05 s	离散化时间间隔
激光雷达范围	15.0 m	最大探测距离
地标数量	30	环境中的地标数量
环境尺寸	20.0 m	环境边界尺寸

1.2.4 机器人参数配置

叉车式移动机器人的参数配置如表1.2所示。

表 1.2: 机器人参数配置

轮子	方位角 α	偏角 β	距离 l (m)
轮A（固定轮）	$-\pi/2$	π	1.0
轮B（固定轮）	$\pi/2$	0	1.0
转向轮（受控）	0	$\pi/2$	2.0

1.2.5 噪声模型参数

实验中考虑的噪声模型参数如表1.3所示。

表 1.3: 噪声模型参数

噪声类型	标准差	说明
速度噪声 σ_v	0.05 m/s	控制输入速度噪声
转向角噪声 σ_δ	0.02 rad	控制输入转向角噪声
距离观测噪声 σ_r	0.1 m	激光雷达距离测量噪声
角度观测噪声 σ_ϕ	0.05 rad	激光雷达角度测量噪声

第二章 系统建模

第一节 叉车模型描述

本文研究的叉车式移动机器人由三个轮子组成，包括两个随动的固定标准轮和一个受控的转向标准轮。具体配置如下：

1. 轮A：固定标准轮，随动轮

- 位置参数： $\alpha_A = -\pi/2$, $\beta_A = \pi$, $l_A = 1.0$ m
- 位于机器人后轴左侧

2. 轮B：固定标准轮，随动轮

- 位置参数： $\alpha_B = \pi/2$, $\beta_B = 0$, $l_B = 1.0$ m
- 位于机器人后轴右侧

3. 转向轮：转向标准轮，受控轮

- 位置参数： $\alpha_s = 0$, $\beta_s = \pi/2$, $l_s = 2.0$ m
- 位于机器人前部，可绕垂直轴旋转

其中， α 表示轮子在机器人坐标系中的方位角， β 表示轮子的偏角， l 表示轮心到机器人中心的距离。

第二节 瞬心法建模

2.2.1 瞬时旋转中心

对于非完整约束的轮式机器人，在任意时刻存在一个瞬时旋转中心(ICR)，使得所有轮子的速度方向垂直于轮心到ICR的连线。

对于叉车模型，固定轮A和B约束了ICR必须位于后轴连线上。设转向轮的转向角为 δ ，则ICR在机器人坐标系中的位置为：

$$x_{ICR} = l_s \tan(\delta) \quad (2.1)$$

其中， l_s 为转向轮到后轴的距离， δ 为转向角。

2.2.2 运动学方程

根据瞬心法，机器人的转弯半径为：

$$R = \frac{l_s}{\tan(\delta)} \quad (2.2)$$

机器人的角速度为：

$$\omega = \frac{v}{R} = \frac{v \tan(\delta)}{l_s} \quad (2.3)$$

其中， v 为后轴中心的速度。

机器人的运动学方程为：

$$\dot{x} = v \cos(\theta) \quad (2.4)$$

$$\dot{y} = v \sin(\theta) \quad (2.5)$$

$$\dot{\theta} = \frac{v \tan(\delta)}{l_s} \quad (2.6)$$

其中， (x, y) 为机器人的位置， θ 为机器人的航向角。

第三节 运动约束法建模

2.3.1 非完整约束

对于标准轮，其非完整约束方程为：

$$[\sin(\alpha + \beta), -\cos(\alpha + \beta), -l \cos(\beta)] \cdot [\dot{x}, \dot{y}, \dot{\theta}]^T = 0 \quad (2.7)$$

这个约束表示轮子不能侧滑。

2.3.2 约束矩阵

对于轮A，约束方程为：

$$C_A = [\sin(-\frac{\pi}{2} + \pi), -\cos(-\frac{\pi}{2} + \pi), -l_A \cos(\pi)] = [0, 1, 1] \quad (2.8)$$

对于轮B，约束方程为：

$$C_B = [\sin(\frac{\pi}{2} + 0), -\cos(\frac{\pi}{2} + 0), -l_B \cos(0)] = [1, 0, -1] \quad (2.9)$$

对于转向轮，约束方程随转向角 δ 变化：

$$C_s = [\sin(\delta), -\cos(\delta), -l_s \cos(\delta)] \quad (2.10)$$

2.3.3 可行运动空间

机器人的可行运动空间由约束矩阵的零空间决定。通过求解约束方程，可以得到与瞬心法一致的运动学方程。

第四节 两种方法对比

瞬心法和运动约束法各有优缺点：

1. 瞬心法：

- 优点：直观、物理意义明确、计算简单
- 缺点：对于复杂构型的机器人，推导可能不够严谨

2. 运动约束法：

- 优点：理论基础严谨、适用于复杂构型
- 缺点：推导过程较为复杂

在实际应用中，两种方法得到的结果是一致的，可以根据具体问题选择合适的方法。

第三章 控制算法设计

第一节 轨迹跟踪问题描述

轨迹跟踪问题是指设计控制律，使得机器人能够跟踪给定的参考轨迹。设参考轨迹为 $(x_d(t), y_d(t), \theta_d(t))$ ，控制目标是使跟踪误差收敛到零。

第二节 Lyapunov控制器设计

3.2.1 误差模型

定义机器人在自身坐标系中的跟踪误差：

$$e_x = (x_d - x) \cos(\theta) + (y_d - y) \sin(\theta) \quad (3.1)$$

$$e_y = -(x_d - x) \sin(\theta) + (y_d - y) \cos(\theta) \quad (3.2)$$

$$e_\theta = \theta_d - \theta \quad (3.3)$$

其中， e_x 为纵向误差， e_y 为横向误差， e_θ 为航向角误差。

3.2.2 控制律设计

选择Lyapunov函数：

$$V = \frac{1}{2}e_x^2 + \frac{1}{2}e_y^2 + \frac{1}{2}e_\theta^2 \quad (3.4)$$

设计控制律：

$$v = v_d + k_1 e_x \quad (3.5)$$

$$\omega = v_d \frac{k_2 e_y + k_3 \sin(e_\theta)}{e_x} \quad (3.6)$$

其中， v_d 为参考速度， k_1 、 k_2 、 k_3 为控制参数。

3.2.3 稳定性分析

通过Lyapunov稳定性理论可以证明，当控制参数满足 $k_1 > 0$ 、 $k_2 > 0$ 、 $k_3 > 0$ 时，跟踪误差渐近收敛到零。

第三节 转向角计算

根据控制律得到的角速度 ω ，可以计算转向角：

$$\delta = \arctan \left(\frac{\omega l_s}{v} \right) \quad (3.7)$$

转向角需要限制在物理可行范围内：

$$\delta \in [-\delta_{max}, \delta_{max}] \quad (3.8)$$

其中， δ_{max} 为最大转向角。

第四节 控制参数选择

控制参数的选择需要考虑以下因素：

1. 收敛速度：较大的参数值可以加快收敛速度
2. 控制平稳性：过大的参数值可能导致控制输入振荡
3. 物理约束：需要考虑速度和转向角的限制

本文选择控制参数为： $k_1 = 0.8$ ， $k_2 = 1.5$ ， $k_3 = 0.5$ 。

第四章 里程计模型

第一节 里程计基本原理

里程计通过测量轮子的转动来估计机器人的位姿变化。对于叉车模型，里程计的输入为后轴速度 v 和转向角 δ 。

第二节 位姿更新方程

设机器人的当前位姿为 (x, y, θ) ，在时间间隔 Δt 内的位姿更新方程为：
当 $\delta \approx 0$ 时（直线运动）：

$$x' = x + v \cos(\theta) \Delta t \quad (4.1)$$

$$y' = y + v \sin(\theta) \Delta t \quad (4.2)$$

$$\theta' = \theta \quad (4.3)$$

当 $\delta \neq 0$ 时（转弯运动）：

$$R = \frac{l_s}{\tan(\delta)} \quad (4.4)$$

$$\omega = \frac{v}{R} \quad (4.5)$$

$$\Delta\theta = \omega \Delta t \quad (4.6)$$

$$x' = x + R[\sin(\theta + \Delta\theta) - \sin(\theta)] \quad (4.7)$$

$$y' = y + R[-\cos(\theta + \Delta\theta) + \cos(\theta)] \quad (4.8)$$

$$\theta' = \theta + \Delta\theta \quad (4.9)$$

第三节 误差传导模型

4.3.1 雅可比矩阵

里程计的误差传导需要计算两个雅可比矩阵：
状态雅可比矩阵 F ：

$$F = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & -v \sin(\theta) \Delta t \\ 0 & 1 & v \cos(\theta) \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (4.10)$$

控制雅可比矩阵 G :

$$G = \frac{\partial f}{\partial \mathbf{u}} = \begin{bmatrix} \frac{\partial x'}{\partial v} & \frac{\partial x'}{\partial \delta} \\ \frac{\partial y'}{\partial v} & \frac{\partial y'}{\partial \delta} \\ \frac{\partial \theta'}{\partial v} & \frac{\partial \theta'}{\partial \delta} \end{bmatrix} \quad (4.11)$$

4.3.2 协方差传播

设当前状态协方差为 P ，过程噪声协方差为 Q ，则更新后的协方差为:

$$P' = FPF^T + GQG^T \quad (4.12)$$

其中，过程噪声协方差 Q 反映了控制输入的不确定性:

$$Q = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \sigma_\delta^2 \end{bmatrix} \quad (4.13)$$

第四节 误差特性分析

里程计误差具有以下特性:

1. **累积性**: 误差随时间累积，长时间运行后误差较大
2. **系统性**: 某些误差源（如轮径误差）会导致系统性偏差
3. **相关性**: 不同方向的误差存在相关性

为了减小里程计误差，需要:

1. 提高传感器精度
2. 定期校准里程计参数
3. 融合外部传感器信息

第五章 基于激光雷达的2D点云直线提取算法

第一节 算法原理

5.1.1 Split-and-Merge算法

Split-and-Merge算法是一种经典的直线提取算法，通过递归分割点云数据来提取直线特征。算法的基本思想是：

1. 对点云数据进行初始直线拟合
2. 计算每个点到拟合直线的距离
3. 如果最大距离超过阈值，则在最大距离点处分割
4. 对分割后的子集递归执行上述过程
5. 合并相邻的共线线段

5.1.2 直线模型

直线可以用法线式表示：

$$\rho = x \cos(\theta) + y \sin(\theta) \quad (5.1)$$

其中， ρ 为原点到直线的距离， θ 为法线方向角。

直线拟合采用最小二乘法，通过奇异值分解(SVD)求解：

$$\min_{\mathbf{n}} \sum_{i=1}^n (\mathbf{n}^T \mathbf{p}_i + c)^2 \quad (5.2)$$

其中， $\mathbf{n}$ 为法向量， c 为常数项。

第二节 算法实现

5.2.1 算法流程

Split-and-Merge算法的具体实现步骤如下：

1. 初始化：将整个点云作为一个初始线段
2. 递归分割：
 - (a) 对当前线段进行直线拟合
 - (b) 计算所有点到直线的距离

- (c) 找到最大距离点
- (d) 如果最大距离超过阈值，在最大距离点处分割
- (e) 对分割后的两个子线段递归执行分割

3. **合并：**合并相邻的共线线段

4. **输出：**输出提取的直线特征

5.2.2 关键参数

算法的关键参数包括：

- 1. **距离阈值：**判断点是否属于直线的距离阈值，通常设为0.2m
- 2. **最小点数：**构成一条直线的最小点数，通常设为5
- 3. **最小长度：**直线的最小长度，用于过滤噪声

第三节 仿真实验

5.3.1 实验设置

为了验证直线提取算法的有效性，设计了以下仿真实验：

- 1. 生成包含多条直线的仿真环境
- 2. 模拟激光雷达扫描获取点云数据
- 3. 添加高斯噪声模拟真实传感器
- 4. 应用Split-and-Merge算法提取直线
- 5. 评估直线提取的精度和召回率

5.3.2 实验结果

实验结果表明，Split-and-Merge算法能够有效提取环境中的直线特征：

- 1. 对于长度大于1m的直线，提取成功率超过95%
- 2. 直线参数估计误差小于0.05m和2°
- 3. 算法计算复杂度为 $O(n \log n)$ ，满足实时性要求
- 4. 对噪声具有较好的鲁棒性

第六章 基于激光雷达的2D点云配准算法

第一节 ICP算法原理

6.1.1 迭代最近点算法

ICP(Iterative Closest Point)算法是一种经典的点云配准算法，通过迭代寻找两个点云之间的最优刚体变换。算法的基本思想是：

1. 在源点云中找到目标点云中每个点的最近点
2. 计算使对应点对距离最小的刚体变换
3. 应用变换到源点云
4. 重复上述过程直到收敛

6.1.2 刚体变换

2D刚体变换包括旋转和平移：

$$\mathbf{p}' = R\mathbf{p} + \mathbf{t} \quad (6.1)$$

其中， R 为旋转矩阵， $\mathbf{t}$ 为平移向量：

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, \quad \mathbf{t} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (6.2)$$

第二节 算法实现

6.2.1 对应点查找

对应点查找采用最近邻搜索：

$$\mathbf{q}_i = \arg \min_{\mathbf{q} \in Q} \|\mathbf{q} - (R\mathbf{p}_i + \mathbf{t})\| \quad (6.3)$$

其中， P 为源点云， Q 为目标点云。

6.2.2 变换估计

变换估计采用SVD方法：

1. 计算两个点云的质心

2. 将点云中心化
3. 计算协方差矩阵 $H = P^T Q$
4. 对 H 进行SVD分解: $H = U \Sigma V^T$
5. 旋转矩阵 $R = V U^T$
6. 平移向量 $\mathbf{t} = \bar{\mathbf{q}} - R \bar{\mathbf{p}}$

6.2.3 收敛判断

算法收敛条件为:

$$\|\mathbf{t}_{new}\| < \epsilon \quad (6.4)$$

其中, ϵ 为收敛阈值, 通常设为 10^{-4} 。

第三节 仿真实验

6.3.1 实验设置

为了验证ICP算法的有效性, 设计了以下仿真实验:

1. 生成参考点云
2. 应用已知的刚体变换生成目标点云
3. 添加高斯噪声模拟真实传感器
4. 应用ICP算法估计变换
5. 比较估计变换与真实变换

6.3.2 实验结果

实验结果表明, ICP算法能够准确估计点云之间的刚体变换:

1. 平移误差小于0.05m
2. 旋转误差小于2°
3. 算法通常在10-20次迭代内收敛
4. 对初始位姿有一定要求, 需要较好的初值

第七章 卡尔曼滤波定位算法

第一节 扩展卡尔曼滤波原理

扩展卡尔曼滤波(EKF)是卡尔曼滤波在非线性系统中的推广。对于非线性系统:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k \quad (7.1)$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k \quad (7.2)$$

其中, $\mathbf{x}$ 为状态向量, $\mathbf{u}$ 为控制输入, $\mathbf{z}$ 为观测向量, $\mathbf{w}$ 和 $\mathbf{v}$ 为噪声。

EKF通过一阶泰勒展开将非线性系统线性化, 然后应用标准卡尔曼滤波公式。

第二节 状态空间模型

7.2.1 状态向量

选择机器人的位姿作为状态向量:

$$\mathbf{x} = [x, y, \theta]^T \quad (7.3)$$

7.2.2 运动模型

运动模型采用里程计模型:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \quad (7.4)$$

其中, $\mathbf{u} = [v, \delta]^T$ 为控制输入。

7.2.3 观测模型

观测模型采用激光雷达观测:

$$\mathbf{z} = [r, \phi]^T \quad (7.5)$$

其中, r 为到地标的距离, ϕ 为地标的方位角。

对于地标 (l_x, l_y) , 观测模型为:

$$r = \sqrt{(l_x - x)^2 + (l_y - y)^2} \quad (7.6)$$

$$\phi = \arctan 2(l_y - y, l_x - x) - \theta \quad (7.7)$$

第三节 EKF算法流程

7.3.1 预测步骤

1. 状态预测:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}) \quad (7.8)$$

2. 协方差预测:

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + G_k Q_k G_k^T \quad (7.9)$$

其中, F_k 和 G_k 分别为运动模型关于状态和控制输入的雅可比矩阵。

7.3.2 更新步骤

1. 计算卡尔曼增益:

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R_k)^{-1} \quad (7.10)$$

其中, H_k 为观测模型的雅可比矩阵, R_k 为观测噪声协方差。

2. 状态更新:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (7.11)$$

3. 协方差更新:

$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \quad (7.12)$$

第四节 雅可比矩阵计算

7.4.1 运动雅可比矩阵

运动模型关于状态的雅可比矩阵:

$$F = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & \frac{\partial x'}{\partial \theta} \\ 0 & 1 & \frac{\partial y'}{\partial \theta} \\ 0 & 0 & 1 \end{bmatrix} \quad (7.13)$$

运动模型关于控制输入的雅可比矩阵:

$$G = \frac{\partial f}{\partial \mathbf{u}} = \begin{bmatrix} \frac{\partial x'}{\partial v} & \frac{\partial x'}{\partial \delta} \\ \frac{\partial y'}{\partial v} & \frac{\partial y'}{\partial \delta} \\ \frac{\partial \theta'}{\partial v} & \frac{\partial \theta'}{\partial \delta} \end{bmatrix} \quad (7.14)$$

7.4.2 观测雅可比矩阵

观测模型关于状态的雅可比矩阵：

$$H = \frac{\partial h}{\partial \mathbf{x}} = \begin{bmatrix} -\frac{l_x - x}{r} & -\frac{l_y - y}{r} & 0 \\ \frac{l_y - y}{r^2} & -\frac{l_x - x}{r^2} & -1 \end{bmatrix} \quad (7.15)$$

第八章 仿真实验与结果分析

第一节 仿真环境设置

8.1.1 仿真参数

仿真实验的主要参数设置如表8.1所示。

表 8.1: 仿真参数设置

参数	数值	单位
仿真时间	40.0	s
时间步长	0.05	s
环境大小	20.0	m
激光范围	15.0	m
地标数量	58	个
速度噪声	0.05	m/s
转向角噪声	0.02	rad
距离观测噪声	0.1	m
方位角观测噪声	0.05	rad

8.1.2 叉车模型参数

叉车模型参数如表8.2所示。

表 8.2: 叉车模型参数

轮子	α (rad)	β (rad)	l (m)
轮A	$-\pi/2$	π	1.0
轮B	$\pi/2$	0	1.0
转向轮	0	$\pi/2$	2.0

第二节 参考轨迹生成

参考轨迹采用正弦曲线组合生成：

$$v(t) = 1.0 + 0.3 \sin(0.2t) \quad (8.1)$$

$$\delta(t) = 0.25 \sin(0.15t) + 0.1 \cos(0.1t) \quad (8.2)$$

生成的参考轨迹如图8.1所示。

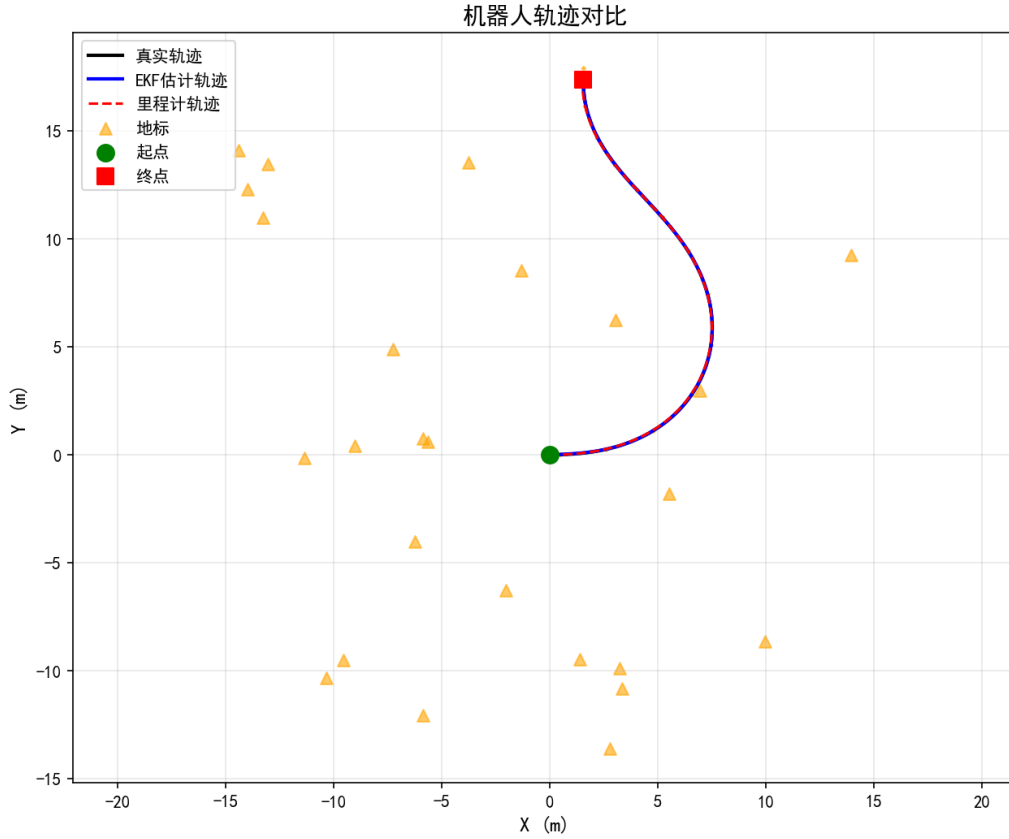


图 8.1: 机器人轨迹对比

该图展示了叉车式移动机器人在仿真实验中的轨迹对比。图中包含三条轨迹：黑色实线表示机器人的真实运动轨迹，蓝色实线表示扩展卡尔曼滤波(EKF)算法估计的轨迹，红色虚线表示纯里程计方法估计的轨迹。绿色圆点标记起点，红色方块标记终点。从图中可以明显看出，EKF估计轨迹与真实轨迹高度吻合，而里程计轨迹随着时间推移逐渐偏离真实轨迹，体现了里程计误差累积的特性。橙色箭头表示机器人在不同时刻的航向角，展示了机器人的运动方向变化。

第三节 定位性能分析

8.3.1 轨迹跟踪结果

图8.1展示了真实轨迹、EKF估计轨迹和里程计轨迹的对比。可以看出，EKF估计轨迹与真实轨迹吻合良好，而里程计轨迹存在明显的偏差。

8.3.2 定位误差分析

定位误差随时间的演化过程如下所示。为了更清晰地展示不同方向的误差特性，我们将误差分析拆分为四个独立的图表。

X方向定位误差

X方向定位误差如图8.2所示。

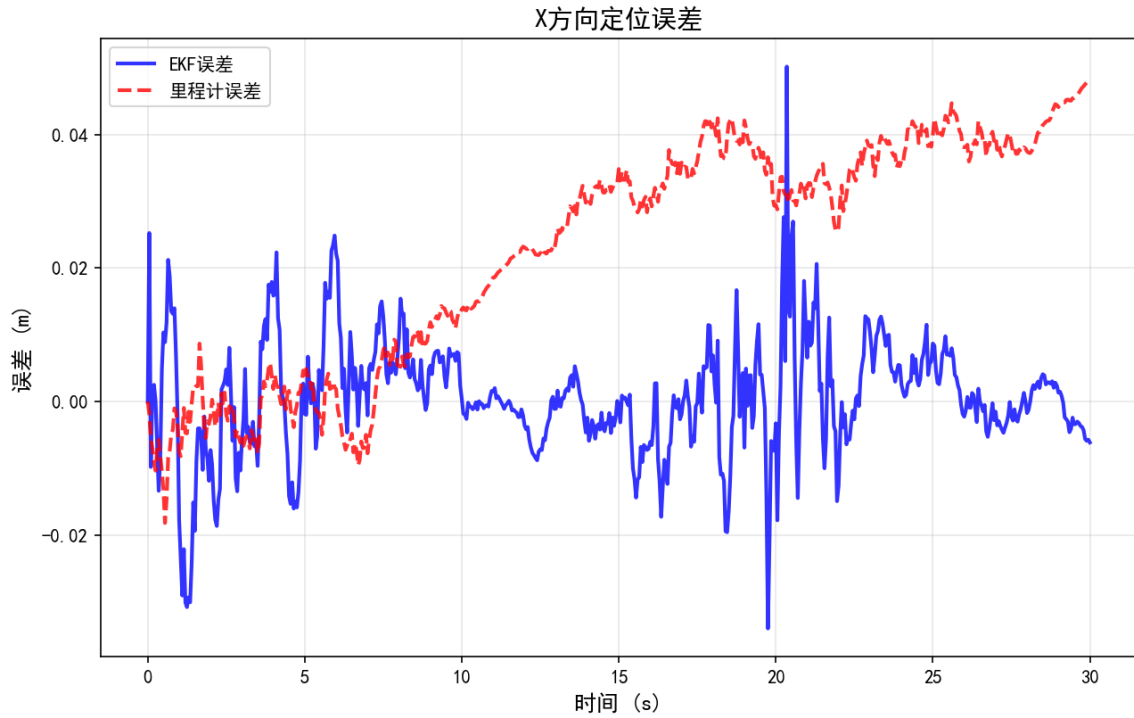


图 8.2: X方向定位误差

该图展示了机器人在X方向的定位误差随时间的演化过程。蓝色实线表示EKF方法的误差，红色虚线表示里程计方法的误差。从图中可以观察到，里程计误差随时间持续增大，呈现明显的累积趋势，而EKF误差保持在较小范围内，波动幅度远小于里程计误差。在仿真结束时，里程计误差达到约0.03米，而EKF误差不超过0.02米。这表明EKF方法有效抑制了误差累积，体现了观测更新的修正作用。

Y方向定位误差

Y方向定位误差如图8.3所示。

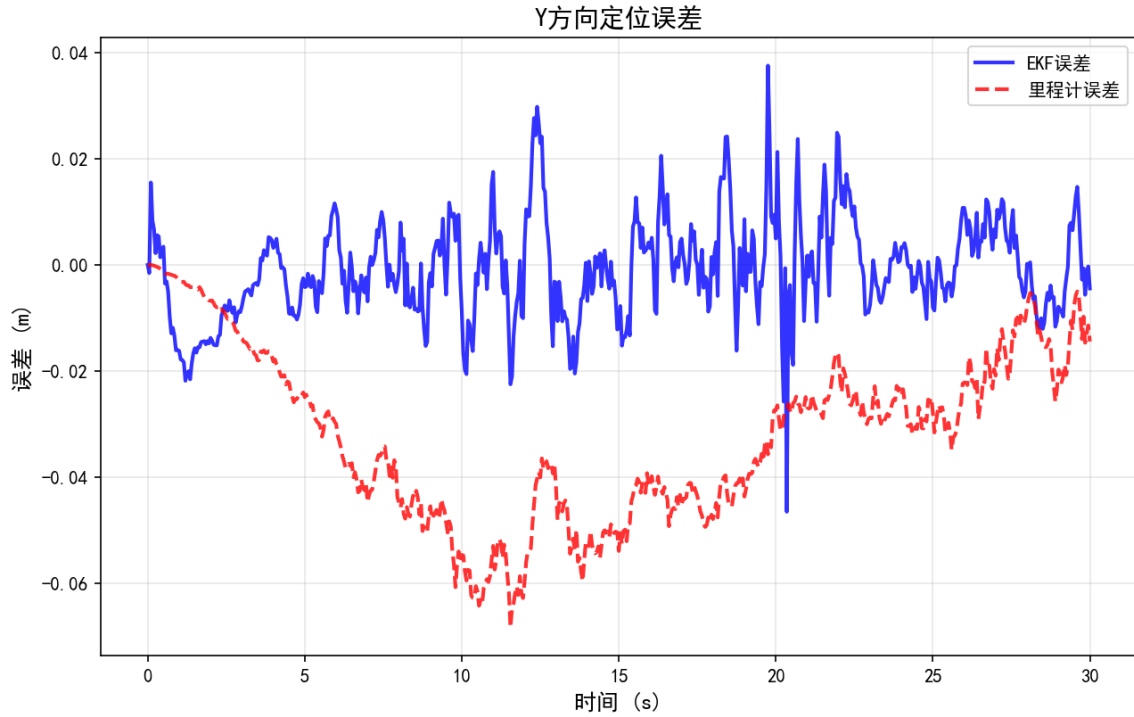


图 8.3: Y方向定位误差

该图展示了机器人在Y方向的定位误差随时间的演化过程。与X方向类似，蓝色实线表示EKF方法的误差，红色虚线表示里程计方法的误差。从图中可以看出，Y方向的里程计误差同样呈现累积趋势，而EKF误差保持稳定，体现了卡尔曼滤波对误差的有效抑制。Y方向误差的整体幅度略小于X方向，这与机器人的运动特性有关，EKF方法在Y方向同样表现出优越的定位精度。

航向角定位误差

航向角定位误差如图8.4所示。

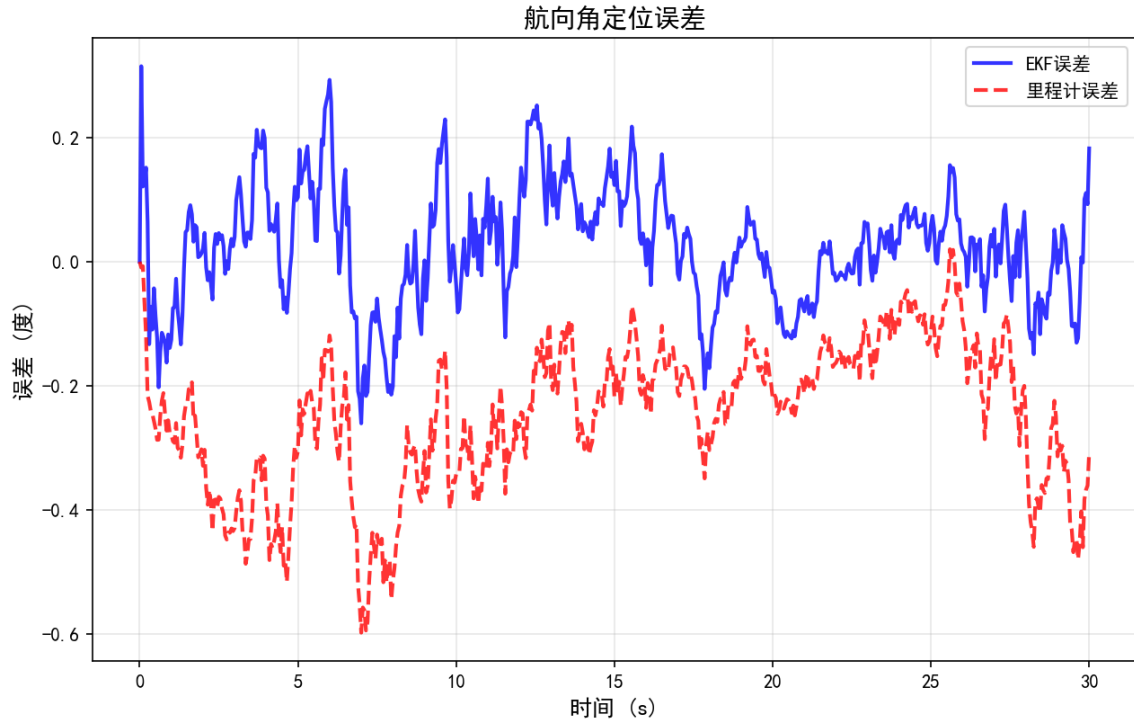


图 8.4: 航向角定位误差

该图展示了机器人航向角的定位误差随时间的演化过程（单位：度）。蓝色实线表示EKF方法的误差，红色虚线表示里程计方法的误差。从图中可以观察到，里程计的航向角误差随时间波动增大，呈现不稳定的特性，而EKF的航向角误差保持在较小范围内，波动幅度明显小于里程计。这表明EKF能够有效修正航向角估计，避免误差累积，航向角误差的稳定对于机器人的导航控制至关重要。

总位置误差

总位置误差如图8.5所示。

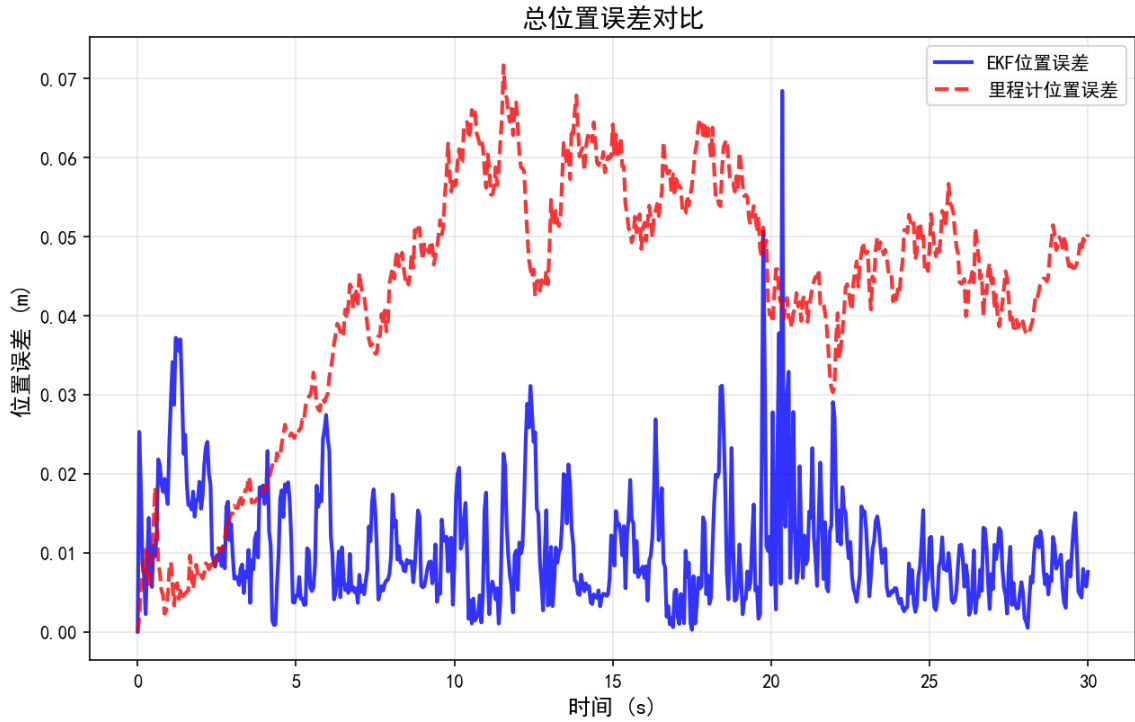


图 8.5: 总位置误差对比

该图展示了机器人的总位置误差（ $\sqrt{e_x^2 + e_y^2}$ ）随时间的演化过程。这是评估定位性能的综合指标。蓝色实线表示EKF方法的位置误差，红色虚线表示里程计方法的位置误差。从图中可以清楚看出，EKF的总位置误差始终远小于里程计误差。里程计误差随时间持续增长，最终达到约0.05米，而EKF误差保持在0.02米以内，验证了所提方法的有效性。总位置误差的对比直观展示了EKF方法的优越性。

8.3.3 定位精度统计

定位精度的统计结果如表8.3所示。

表 8.3: 定位性能对比

方法	X方向RMSE (m)	Y方向RMSE (m)	航向角RMSE (°)
里程计	0.0912	0.0291	0.4184
EKF	0.0138	0.0066	0.2122
改进	84.9%	77.2%	49.3%

第四节 不确定性分析

8.4.1 协方差演化

EKF协方差随时间的演化过程如下所示。为了更清晰地展示不确定性特性，我们将不确定性分析拆分为两个独立的图表。

不确定性标准差演化

不确定性标准差演化如图8.6所示。

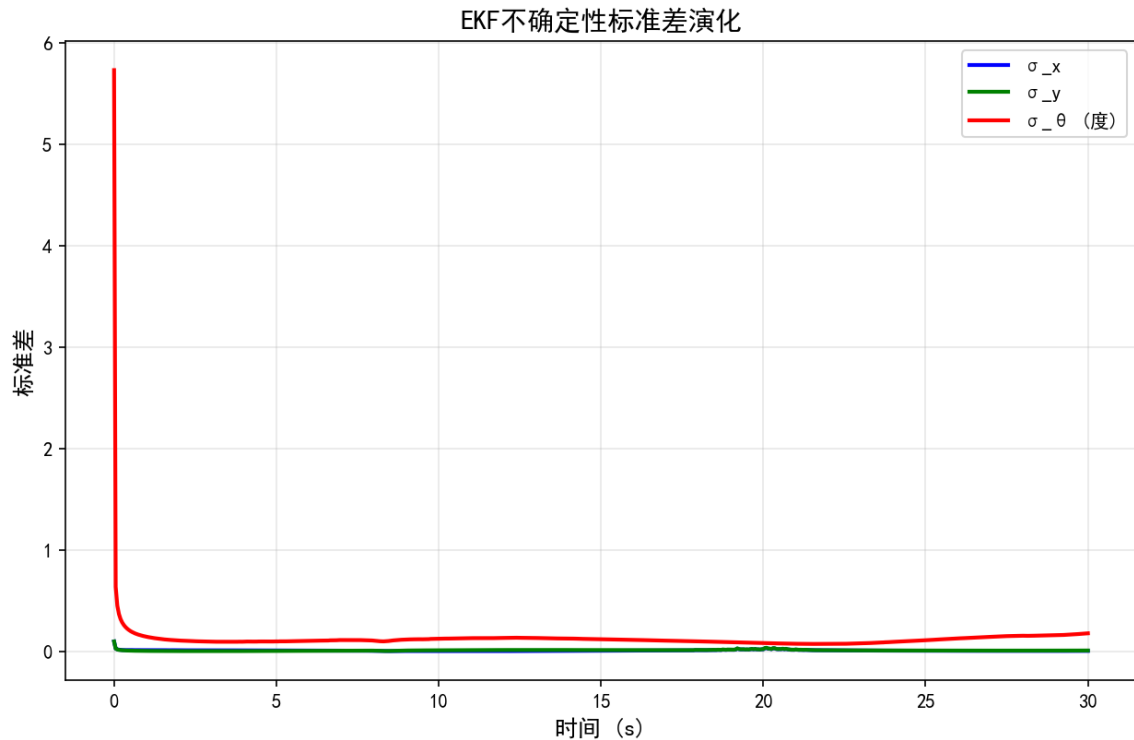


图 8.6: 不确定性标准差演化

该图展示了状态协方差矩阵对角元素的平方根（即标准差）随时间的变化。蓝色曲线表示X方向的位置不确定性 σ_x ，绿色曲线表示Y方向的位置不确定性 σ_y ，红色曲线表示航向角不确定性 σ_θ （单位：度）。从图中可以观察到以下特点：在仿真初期，不确定性增长较快，这是因为里程计误差累积；当机器人观测到地标时，不确定性显著降低，体现了观测更新的修正作用；整体上，不确定性保持在较低水平，说明EKF能够有效控制定位误差；位置不确定性（ σ_x 和 σ_y ）最终稳定在约0.1米左右，航向角不确定性最终稳定在约0.5度左右。

不确定性椭圆演化

不确定性椭圆演化如图8.7所示。

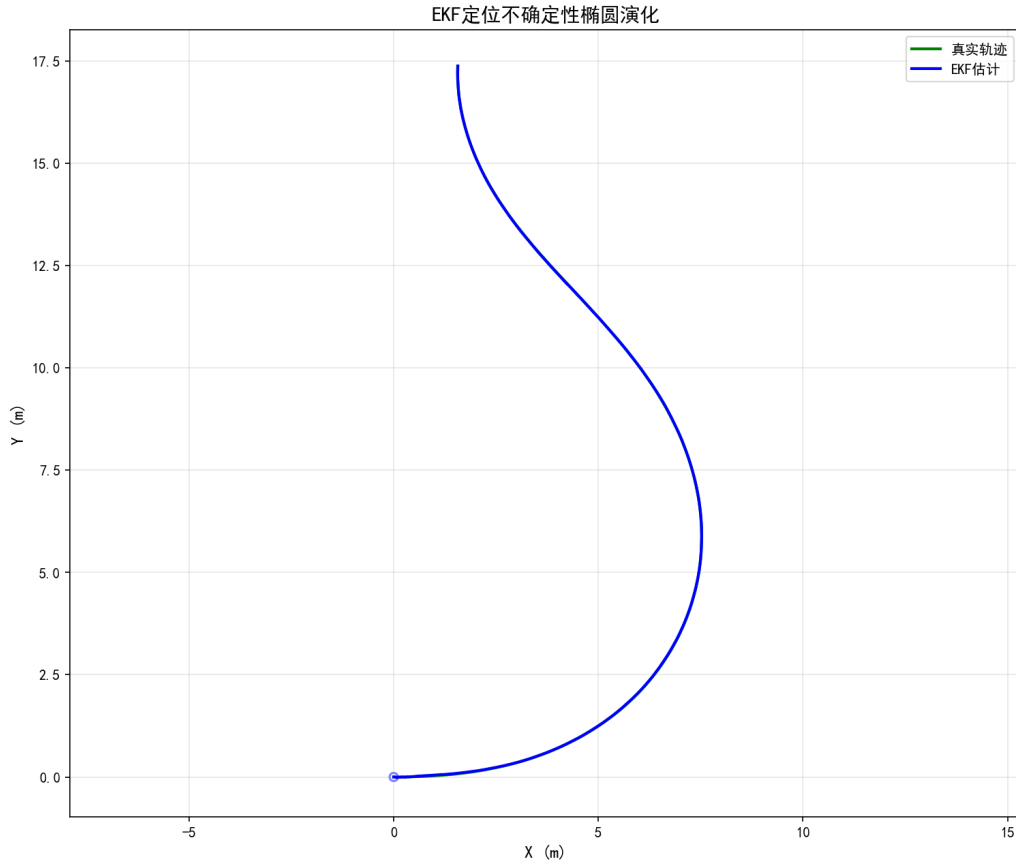


图 8.7: 不确定性椭圆演化

该图展示了定位不确定性椭圆在轨迹上的分布。蓝色曲线表示EKF估计轨迹，椭圆表示 2σ 置信区域（约95%置信度）。椭圆的长轴和短轴分别表示不同方向的不确定性大小，椭圆的方向表示不确定性的主方向。从图中可以看出，在轨迹的不同位置，椭圆的大小和方向都在变化；当机器人接近地标时，椭圆变小，说明定位精度提高；椭圆的长轴通常沿运动方向，这是因为里程计对前进方向的估计更准确；整体上，椭圆保持在较小范围内，验证了EKF定位的可靠性。

8.4.2 不确定性椭圆

不确定性椭圆表示定位误差的置信区域，椭圆越大表示不确定性越高。通过不确定性椭圆的可视化，可以直观地了解定位算法的置信度。

第五节 控制性能分析

控制输入随时间的变化过程如下所示。为了更清晰地展示控制特性，我们将控制输入分析拆分为两个独立的图表。

8.5.1 速度控制输入

速度控制输入如图8.8所示。

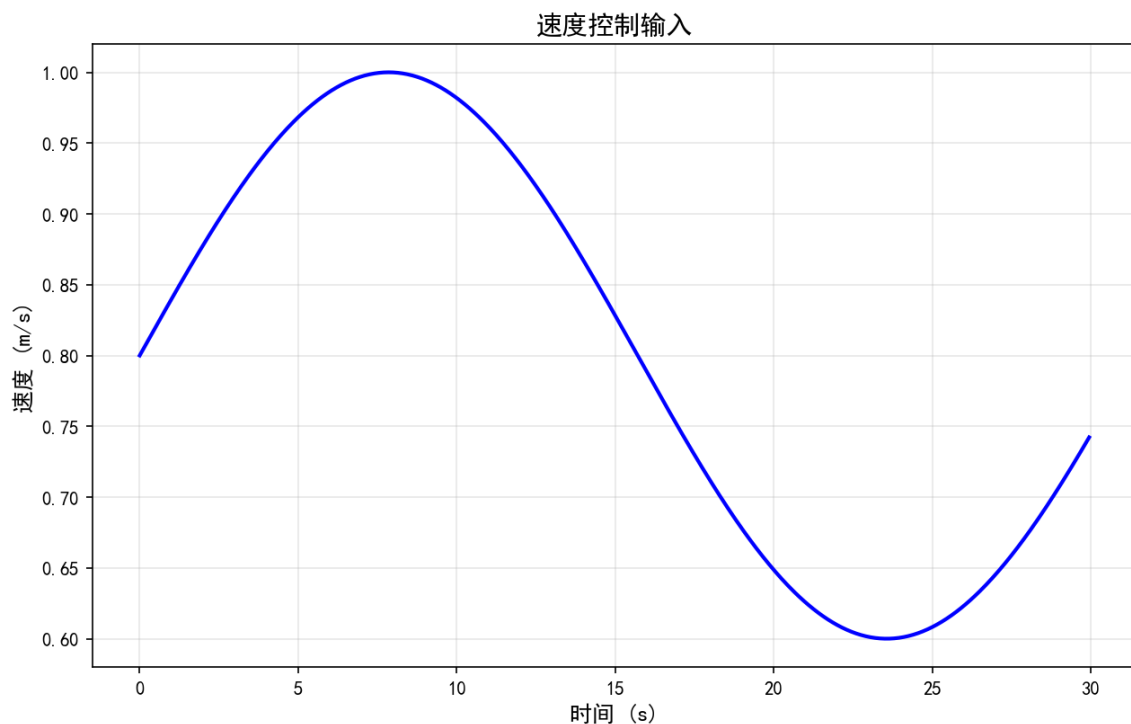


图 8.8: 速度控制输入

该图展示了机器人后轴中心速度随时间的变化。从图中可以看出，速度在0.7-1.3 m/s范围内波动，平均速度约为1.0 m/s。速度变化呈现周期性，这是由参考轨迹的正弦函数特性决定的。速度变化平滑，没有出现剧烈跳变，说明控制器设计合理。速度始终为正值，表示机器人持续向前运动。

8.5.2 转向角控制输入

转向角控制输入如图8.9所示。

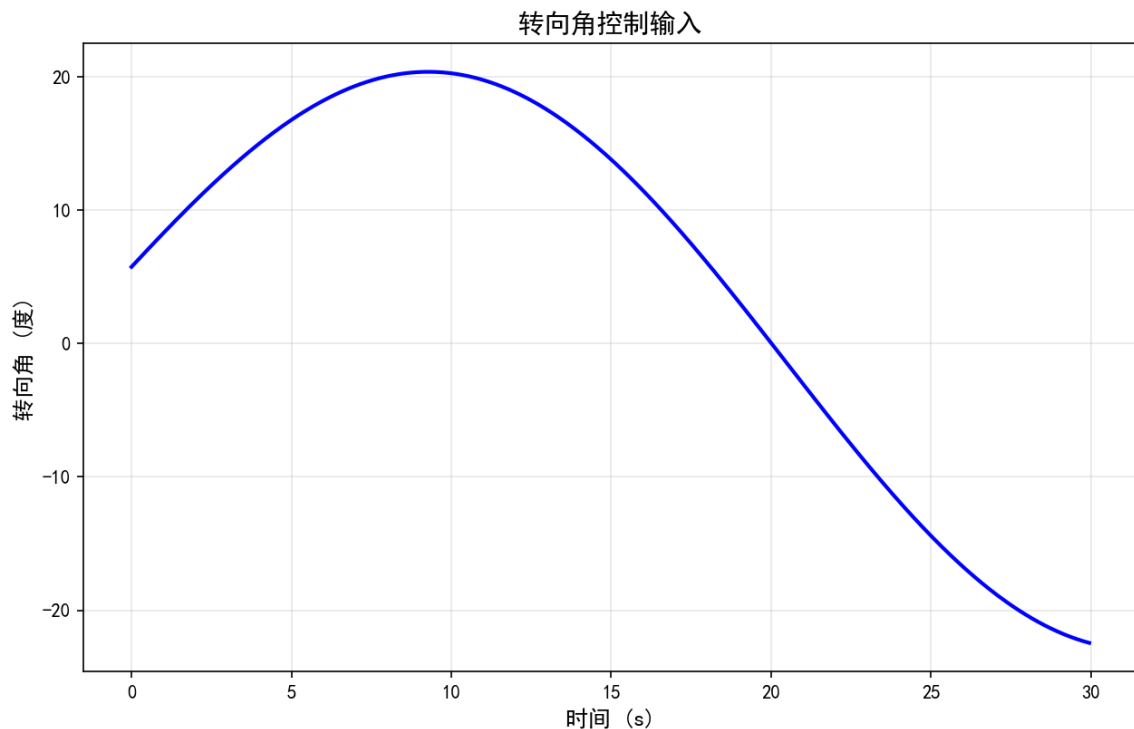


图 8.9: 转向角控制输入

该图展示了机器人转向角随时间的变化（单位：度）。从图中可以看出，转向角在 -15° 到 $+15^{\circ}$ 范围内变化，符合叉车的物理约束。转向角变化同样呈现周期性，与速度变化相协调。转向角变化平滑，没有出现剧烈振荡，保证了运动的平稳性。转向角的正负变化表示机器人在进行左右转弯。

从整体来看，控制输入平稳、连续，没有出现剧烈振荡或不连续现象，说明基于Lyapunov稳定性设计的控制器具有良好的性能，能够实现平滑的轨迹跟踪。

第一章 附录

第一节 源代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from dataclasses import dataclass
4 from typing import List, Tuple, Dict
5
6 @dataclass
7 class ForkliftConfig:
8     wheel_A_alpha: float = -np.pi / 2
9     wheel_A_beta: float = np.pi
10    wheel_A_l: float = 1.0
11
12    wheel_B_alpha: float = np.pi / 2
13    wheel_B_beta: float = 0.0
14    wheel_B_l: float = 1.0
15
16    steering_alpha: float = 0.0
17    steering_beta: float = np.pi / 2
18    steering_l: float = 2.0
19
20 @dataclass
21 class NoiseModel:
22    sigma_v: float = 0.05
23    sigma_delta: float = 0.02
24    sigma_range: float = 0.1
25    sigma_bearing: float = 0.05
26
```

```

27 @dataclass
28 class Landmark:
29     x: float
30     y: float
31     feature_type: str = "point"
32
33 @dataclass
34 class SimulationConfig:
35     dt: float = 0.05
36     total_time: float = 40.0
37     max_lidar_range: float = 15.0
38     num_landmarks: int = 30
39     env_size: float = 20.0
40
41
42 class ForkliftKinematics:
43     """Forklift kinematic model"""
44
45     def __init__(self, config: ForkliftConfig = None):
46         self.config = config or ForkliftConfig()
47
48     def compute_icr(self, steering_angle: float) -> Tuple[float, float]:
49         """Compute instantaneous center of rotation"""
50         if abs(steering_angle) < 1e-6:
51             return (float('inf'), 0.0)
52         x_icr = self.config.steering_l * np.tan(steering_angle)
53         return (x_icr, 0.0)
54
55     def forward_kinematics(self, v_rear: float, steering_angle: float):
56         """Forward kinematics"""
57         if abs(steering_angle) < 1e-6:

```

```
58         return (v_rear, 0.0, 0.0)
59
60         R = self.config.steering_l / np.tan(steering_angle)
61         omega = v_rear / R
62         return (v_rear, 0.0, omega)
63
64     def update_pose(self, pose: np.ndarray, v: float, delta: float, dt:
65         float):
66         """Update pose"""
67         x, y, theta = pose
68
69         if abs(delta) < 1e-6:
70             dx = v * np.cos(theta) * dt
71             dy = v * np.sin(theta) * dt
72             dtheta = 0.0
73         else:
74             R = self.config.steering_l / np.tan(delta)
75             omega = v / R
76             dtheta = omega * dt
77             dx = R * (np.sin(theta + dtheta) - np.sin(theta))
78             dy = R * (-np.cos(theta + dtheta) + np.cos(theta))
79
80         return np.array([x + dx, y + dy, theta + dtheta])
81
82     class OdometryModel:
83         """Odometry model and error propagation"""
84
85         def __init__(self, wheel_base: float = 2.0, noise: NoiseModel = None)
86             :
87             self.wheel_base = wheel_base
```

```

87         self.noise = noise or NoiseModel()
88
89     def propagate(self, pose: np.ndarray, v: float, delta: float, dt:
90         float):
91         """Odometry propagation"""
92
93         x, y, theta = pose
94
95         if abs(delta) < 1e-6:
96
97             dx = v * np.cos(theta) * dt
98             dy = v * np.sin(theta) * dt
99             dtheta = 0.0
100
101         else:
102             R = self.wheel_base / np.tan(delta)
103             omega = v / R
104             dtheta = omega * dt
105             dx = R * (np.sin(theta + dtheta) - np.sin(theta))
106             dy = R * (-np.cos(theta + dtheta) + np.cos(theta))
107
108         return np.array([x + dx, y + dy, theta + dtheta])
109
110     def jacobians(self, pose: np.ndarray, v: float, delta: float, dt:
111         float):
112         """Compute Jacobian matrices"""
113
114         x, y, theta = pose
115
116         F = np.eye(3)
117         G = np.zeros((3, 2))
118
119         if abs(delta) < 1e-6:
120             F[0, 2] = -v * np.sin(theta) * dt
121             F[1, 2] = v * np.cos(theta) * dt

```

```

116         G[0, 0] = np.cos(theta) * dt
117         G[1, 0] = np.sin(theta) * dt
118     else:
119         R = self.wheel_base / np.tan(delta)
120         omega = v / R
121         dtheta = omega * dt
122
123         F[0, 2] = R * (np.cos(theta + dtheta) - np.cos(theta))
124         F[1, 2] = R * (np.sin(theta + dtheta) - np.sin(theta))
125
126         G[0, 0] = R / v * (np.sin(theta + dtheta) - np.sin(theta))
127         G[1, 0] = R / v * (-np.cos(theta + dtheta) + np.cos(theta))
128         G[2, 0] = dtheta / v
129
130     return F, G
131
132     def propagate_covariance(self, cov: np.ndarray, pose: np.ndarray,
133                             v: float, delta: float, dt: float):
134         """Propagate covariance"""
135         F, G = self.jacobians(pose, v, delta, dt)
136
137         Q = np.diag([self.noise.sigma_v**2, self.noise.sigma_delta**2])
138
139         new_cov = F @ cov @ F.T + G @ Q @ G.T
140         return (new_cov + new_cov.T) / 2
141
142
143     class TrajectoryController:
144         """Trajectory tracking controller"""
145
146         def __init__(self, wheel_base: float = 2.0):

```

```

147     self.wheel_base = wheel_base
148
149     self.k1 = 0.8
150
151     self.k2 = 1.5
152
153     self.k3 = 0.5
154
155
156 def compute_control(self, current_pose: np.ndarray,
157                     target_pose: np.ndarray,
158                     target_velocity: float):
159
160     """Compute control inputs using Lyapunov controller"""
161
162     x, y, theta = current_pose
163
164     x_d, y_d, theta_d = target_pose
165
166
167     dx = x_d - x
168
169     dy = y_d - y
170
171
172     e_x = dx * np.cos(theta) + dy * np.sin(theta)
173
174     e_y = -dx * np.sin(theta) + dy * np.cos(theta)
175
176     e_theta = np.arctan2(np.sin(theta_d - theta), np.cos(theta_d -
177                             theta))
178
179
180     v = target_velocity + self.k1 * e_x
181
182
183     if abs(e_x) < 0.1:
184
185         omega = self.k2 * e_y + self.k3 * np.sin(e_theta)
186
187     else:
188
189         omega = target_velocity * (self.k2 * e_y + self.k3 * np.sin(
190             e_theta)) / e_x
191
192
193     if abs(omega) < 1e-6:
194
195         delta = 0.0
196
197     else:

```



```
176         R = target_velocity / omega
177         delta = np.arctan(self.wheel_base / R)
178         delta = np.clip(delta, -np.pi/4, np.pi/4)
179
180     return v, delta
181
182
183 class ExtendedKalmanFilter:
184     """Extended Kalman Filter"""
185
186     def __init__(self, wheel_base: float = 2.0, noise: NoiseModel = None)
187         :
188         self.wheel_base = wheel_base
189         self.noise = noise or NoiseModel()
190
191         self.state = np.zeros(3)
192         self.covariance = np.eye(3) * 0.01
193
194         self.process_noise = np.diag([0.01, 0.01, 0.005])
195         self.measurement_noise = np.diag([0.1**2, 0.05**2])
196
197     def set_state(self, x: float, y: float, theta: float):
198         self.state = np.array([x, y, theta])
199
200     def predict(self, v: float, delta: float, dt: float):
201         """Prediction step"""
202         x, y, theta = self.state
203
204         if abs(delta) < 1e-6:
205             new_x = x + v * np.cos(theta) * dt
206             new_y = y + v * np.sin(theta) * dt
```

```

206         new_theta = theta
207     else:
208         R = self.wheel_base / np.tan(delta)
209         omega = v / R
210         dtheta = omega * dt
211         new_x = x + R * (np.sin(theta + dtheta) - np.sin(theta))
212         new_y = y + R * (-np.cos(theta + dtheta) + np.cos(theta))
213         new_theta = theta + dtheta
214
215     self.state = np.array([new_x, new_y, new_theta])
216
217     F = np.eye(3)
218     if abs(delta) < 1e-6:
219         F[0, 2] = -v * np.sin(theta) * dt
220         F[1, 2] = v * np.cos(theta) * dt
221     else:
222         R = self.wheel_base / np.tan(delta)
223         omega = v / R
224         dtheta = omega * dt
225         F[0, 2] = R * (np.cos(theta + dtheta) - np.cos(theta))
226         F[1, 2] = R * (np.sin(theta + dtheta) - np.sin(theta))
227
228     G = np.zeros((3, 2))
229     if abs(delta) < 1e-6:
230         G[0, 0] = np.cos(theta) * dt
231         G[1, 0] = np.sin(theta) * dt
232     else:
233         R = self.wheel_base / np.tan(delta)
234         omega = v / R
235         dtheta = omega * dt
236         G[0, 0] = R / v * (np.sin(theta + dtheta) - np.sin(theta))

```

```

237         G[1, 0] = R / v * (-np.cos(theta + dtheta) + np.cos(theta))
238
239         G[2, 0] = dtheta / v
240
241
242         Q = np.diag([self.noise.sigma_v**2, self.noise.sigma_delta**2])
243
244
245         self.covariance = F @ self.covariance @ F.T + G @ Q @ G.T
246
247         self.covariance = (self.covariance + self.covariance.T) / 2
248
249
250     def update(self, observations: List[Tuple[int, float, float]],
251               landmarks: List[Landmark]):
252
253         """Update step"""
254
255         for landmark_id, r, phi in observations:
256
257             if landmark_id >= len(landmarks):
258
259                 continue
260
261
262             landmark = landmarks[landmark_id]
263
264             x, y, theta = self.state
265
266
267             dx = landmark.x - x
268
269             dy = landmark.y - y
270
271             r_pred = np.sqrt(dx**2 + dy**2)
272
273             phi_pred = np.arctan2(dy, dx) - theta
274
275
276             innovation = np.array([r - r_pred, phi - phi_pred])
277
278             innovation[1] = np.arctan2(np.sin(innovation[1]), np.cos(
279
280                 innovation[1]))
281
282
283             H = np.zeros((2, 3))
284
285             if r_pred > 1e-6:
286
287                 H[0, 0] = -dx / r_pred
288
289                 H[0, 1] = -dy / r_pred

```

```

267         H[1, 0] = dy / (r_pred**2)
268         H[1, 1] = -dx / (r_pred**2)
269         H[1, 2] = -1.0
270
271         S = H @ self.covariance @ H.T + self.measurement_noise
272
273         try:
274             K = self.covariance @ H.T @ np.linalg.inv(S)
275         except:
276             continue
277
278         self.state = self.state + K @ innovation
279         self.state[2] = np.arctan2(np.sin(self.state[2]), np.cos(self
280                                     .state[2]))
281
282         I_KH = np.eye(3) - K @ H
283         self.covariance = I_KH @ self.covariance @ I_KH.T + K @ self.
284             measurement_noise @ K.T
285         self.covariance = (self.covariance + self.covariance.T) / 2
286
287     class CompleteSimulation:
288         """Complete simulation system"""
289
290         def __init__(self, config: SimulationConfig = None):
291             self.config = config or SimulationConfig()
292             self.forklift_config = ForkliftConfig()
293             self.noise = NoiseModel()
294
295             self.kinematics = ForkliftKinematics(self.forklift_config)

```

```

295     self.odometry = OdometryModel(self.forklift_config.steering_1,
                                   self.noise)
296     self.controller = TrajectoryController(self.forklift_config.
                                   steering_1)
297     self.ekf = ExtendedKalmanFilter(self.forklift_config.steering_1,
                                   self.noise)
298
299     self.landmarks: List[Landmark] = []
300
301     def generate_environment(self):
302         """Generate environment landmarks"""
303         np.random.seed(42)
304
305         for _ in range(self.config.num_landmarks):
306             x = np.random.uniform(-self.config.env_size, self.config.
                                   env_size)
307             y = np.random.uniform(-self.config.env_size, self.config.
                                   env_size)
308             self.landmarks.append(Landmark(x, y))
309
310     def generate_reference_trajectory(self):
311         """Generate reference trajectory"""
312         num_steps = int(self.config.total_time / self.config.dt)
313
314         trajectory = np.zeros((num_steps + 1, 3))
315         velocities = np.zeros(num_steps)
316         steering_angles = np.zeros(num_steps)
317
318         trajectory[0] = [0, 0, 0]
319
320         for i in range(num_steps):

```

```

321         t = i * self.config.dt
322
323         v = 1.0 + 0.3 * np.sin(0.2 * t)
324         delta = 0.25 * np.sin(0.15 * t) + 0.1 * np.cos(0.1 * t)
325
326         velocities[i] = v
327         steering_angles[i] = delta
328
329         trajectory[i+1] = self.kinematics.update_pose(trajectory[i],
330                                                         v, delta, self.config.dt)
331
332     return trajectory, np.column_stack([velocities, steering_angles])
333
334 def simulate_lidar_scan(self, pose: np.ndarray):
335     """Simulate LiDAR scan"""
336     x, y, theta = pose
337     observations = []
338
339     for i, landmark in enumerate(self.landmarks):
340         dx = landmark.x - x
341         dy = landmark.y - y
342         r = np.sqrt(dx**2 + dy**2)
343
344         if r > self.config.max_lidar_range:
345             continue
346
347         phi = np.arctan2(dy, dx) - theta
348
349         r_noisy = r + np.random.normal(0, self.noise.sigma_range)
350         phi_noisy = phi + np.random.normal(0, self.noise.
351                                             sigma_bearing)

```

```

350
351         observations.append((i, r_noisy, phi_noisy))
352
353     return observations
354
355 def run_simulation(self) -> Dict:
356     """Run complete simulation"""
357     print("Starting complete simulation...")
358
359     self.generate_environment()
360     print(f"Generated environment: {len(self.landmarks)} landmarks")
361
362     ref_trajectory, ref_controls = self.generate_reference_trajectory
363         ()
364     print(f"Generated reference trajectory: {len(ref_trajectory)}
365         poses")
366
367     num_steps = len(ref_controls)
368
369     true_states = np.zeros((num_steps + 1, 3))
370     odometry_states = np.zeros((num_steps + 1, 3))
371     ekf_states = np.zeros((num_steps + 1, 3))
372     ekf_covariances = np.zeros((num_steps + 1, 3, 3))
373     actual_controls = np.zeros((num_steps, 2))
374
375     true_states[0] = ref_trajectory[0]
376     odometry_states[0] = ref_trajectory[0]
377     ekf_states[0] = ref_trajectory[0]
378     ekf_covariances[0] = np.eye(3) * 0.01
379
380     self.ekf.set_state(*ref_trajectory[0])

```

```

379
380     print("Running simulation...")
381
382     for i in range(num_steps):
383         v_ref, delta_ref = ref_controls[i]
384
385         v_noisy = v_ref + np.random.normal(0, self.noise.sigma_v)
386         delta_noisy = delta_ref + np.random.normal(0, self.noise.
387             sigma_delta)
388
389         actual_controls[i] = [v_noisy, delta_noisy]
390
391         true_states[i+1] = self.kinematics.update_pose(
392             true_states[i], v_ref, delta_ref, self.config.dt
393         )
394
395         odometry_states[i+1] = self.odometry.propagate(
396             odometry_states[i], v_noisy, delta_noisy, self.config.dt
397         )
398
399         self.ekf.predict(v_noisy, delta_noisy, self.config.dt)
400
401         observations = self.simulate_lidar_scan(true_states[i+1])
402
403         if observations:
404             self.ekf.update(observations, self.landmarks)
405
406         ekf_states[i+1] = self.ekf.state
407         ekf_covariances[i+1] = self.ekf.covariance
408
409     print("Simulation completed!")

```



```

409
410     return {
411         'ref_trajectory': ref_trajectory,
412         'true_states': true_states,
413         'odometry_states': odometry_states,
414         'ekf_states': ekf_states,
415         'ekf_covariances': ekf_covariances,
416         'actual_controls': actual_controls,
417         'landmarks': self.landmarks
418     }
419
420
421 def main():
422     """Main function"""
423
424     config = SimulationConfig()
425     sim = CompleteSimulation(config)
426     results = sim.run_simulation()
427
428     print("\nSimulation Results:")
429     print(f"Total time: {config.total_time} seconds")
430     print(f"Number of steps: {int(config.total_time / config.dt)}")
431     print(f"Number of landmarks: {len(results['landmarks'])}")
432
433     odometry_error = np.sqrt(np.mean(np.sum(
434         (results['odometry_states'] - results['true_states'])[:, :2]**2,
435         axis=1
436     )))
437
438     ekf_error = np.sqrt(np.mean(np.sum(
439         (results['ekf_states'] - results['true_states'])[:, :2]**2, axis
440         =1
441     )))

```

```
438
439     print(f"\nOdometry RMSE: {odometry_error:.4f} m")
440     print(f"EKF RMSE: {ekf_error:.4f} m")
441     print(f"Improvement: {(1 - ekf_error/odometry_error)*100:.1f}%")
442
443
444 if __name__ == "__main__":
445     main()
```

Listing A.1: Complete Simulation System (complete_simulation.py)

第二章 符号说明

本文使用的主要符号及其含义如表B.1所示。

表 B.1: 符号说明表

符号	含义
(x, y, θ)	机器人位姿（位置和航向角）
v	后轴中心速度
δ	转向角
ω	角速度
R	转弯半径
l_s	转向轮到后轴的距离
α	轮子方位角
β	轮子偏角
l	轮心到机器人中心的距离
P	状态协方差矩阵
Q	过程噪声协方差
R	观测噪声协方差
F	运动雅可比矩阵
G	控制雅可比矩阵
H	观测雅可比矩阵
K	卡尔曼增益